

```
In [1]: !pip install folium pandas
```

```
Collecting folium
  Using cached folium-0.19.5-py2.py3-none-any.whl.metadata (4.1 kB)
Requirement already satisfied: pandas in c:\users\michael\anaconda3a\lib\site-packages (2.2.3)
Collecting branca>=0.6.0 (from folium)
  Using cached branca-0.8.1-py3-none-any.whl.metadata (1.5 kB)
Requirement already satisfied: jinja2>=2.9 in c:\users\michael\anaconda3a\lib\site-packages (from folium) (3.1.4)
Requirement already satisfied: numpy in c:\users\michael\anaconda3a\lib\site-packages (from folium) (1.26.4)
Requirement already satisfied: requests in c:\users\michael\anaconda3a\lib\site-packages (from folium) (2.32.3)
Requirement already satisfied: xyzservices in c:\users\michael\anaconda3a\lib\site-packages (from folium) (2022.9.0)
Requirement already satisfied: python-dateutil>=2.8.2 in c:\users\michael\anaconda3a\lib\site-packages (from pandas) (2.9.0.post0)
Requirement already satisfied: pytz>=2020.1 in c:\users\michael\anaconda3a\lib\site-packages (from pandas) (2024.1)
Requirement already satisfied: tzdata>=2022.7 in c:\users\michael\anaconda3a\lib\site-packages (from pandas) (2023.3)
Requirement already satisfied: MarkupSafe>=2.0 in c:\users\michael\anaconda3a\lib\site-packages (from jinja2>=2.9->folium) (2.1.3)
Requirement already satisfied: six>=1.5 in c:\users\michael\anaconda3a\lib\site-packages (from python-dateutil>=2.8.2->pandas) (1.16.0)
Requirement already satisfied: charset-normalizer<4,>=2 in c:\users\michael\anaconda3a\lib\site-packages (from requests->folium) (3.3.2)
Requirement already satisfied: idna<4,>=2.5 in c:\users\michael\anaconda3a\lib\site-packages (from requests->folium) (3.7)
Requirement already satisfied: urllib3<3,>=1.21.1 in c:\users\michael\anaconda3a\lib\site-packages (from requests->folium) (2.2.3)
Requirement already satisfied: certifi>=2017.4.17 in c:\users\michael\anaconda3a\lib\site-packages (from requests->folium) (2025.1.31)
Using cached folium-0.19.5-py2.py3-none-any.whl (110 kB)
Using cached branca-0.8.1-py3-none-any.whl (26 kB)
Installing collected packages: branca, folium
Successfully installed branca-0.8.1 folium-0.19.5
```

```
In [2]: import pandas as pd
import folium
```

```
In [3]: # Import folium MarkerCluster plugin
from folium.plugins import MarkerCluster
# Import folium MousePosition plugin
from folium.plugins import MousePosition
# Import folium DivIcon plugin
from folium.features import DivIcon
```

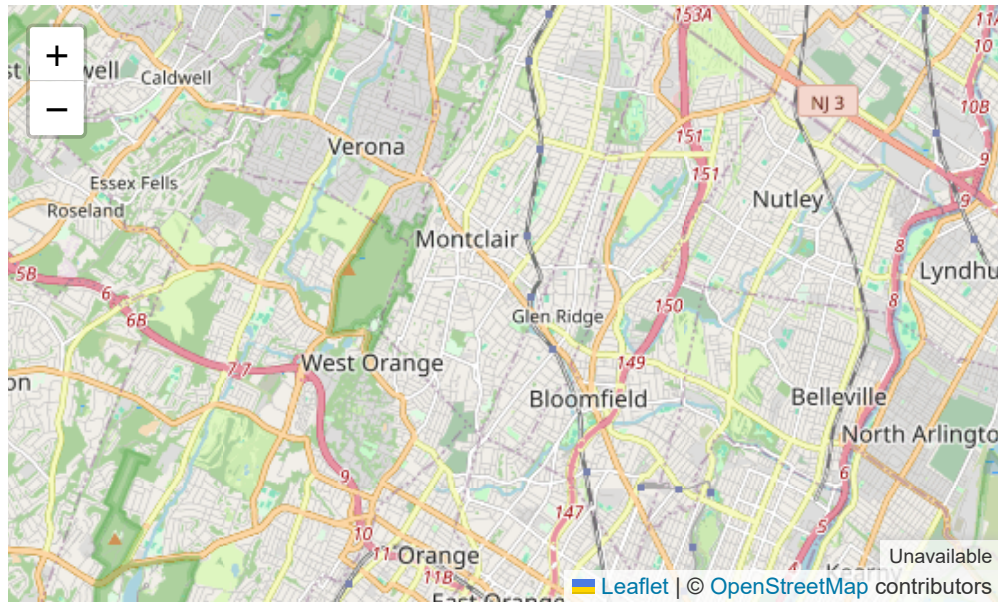
```
In [6]: import pandas as pd

url = 'https://cf-courses-data.s3.us.cloud-object-storage.appdomain.cloud/IBM-DSE101-SPACEX-Data/SpaceX.csv'
spacex_df = pd.read_csv(url)
```

```
spacex_df.head()
```

m

Out[6]:



In [5]: `import pandas as pd`

```
url = 'https://cf-courses-data.s3.us.cloud-object-storage.appdomain.cloud/IBM-DSE103/1/Module%204/data/spacex.csv'
spacex_df = pd.read_csv(url)

spacex_df.head()
```

Out[5]:

	Flight Number	Date	Time (UTC)	Booster Version	Launch Site	Payload	Payload Mass (kg)	Orbit	Customer
0	1	2010-06-04	18:45:00	F9 v1.0 B0003	CCAFS LC-40	Dragon Spacecraft Qualification Unit	0.0	LEO	SpaceX
1	2	2010-12-08	15:43:00	F9 v1.0 B0004	CCAFS LC-40	Dragon demo flight C1, two CubeSats, barrel o...	0.0	LEO (ISS)	NASA (CRS)
2	3	2012-05-22	7:44:00	F9 v1.0 B0005	CCAFS LC-40	Dragon demo flight C2+	525.0	LEO (ISS)	NASA (CRS)
3	4	2012-10-08	0:35:00	F9 v1.0 B0006	CCAFS LC-40	SpaceX CRS-1	500.0	LEO (ISS)	NASA (CRS)
4	5	2013-03-01	15:10:00	F9 v1.0 B0007	CCAFS LC-40	SpaceX CRS-2	677.0	LEO (ISS)	NASA (CRS)

```
In [7]: # Keep only the necessary columns
spacex_df = spacex_df[['Launch Site', 'Lat', 'Long', 'class']]

# Group by launch site and keep the first occurrence
launch_sites_df = spacex_df.groupby(['Launch Site'], as_index=False).first()

# Keep only the columns needed for the map
launch_sites_df = launch_sites_df[['Launch Site', 'Lat', 'Long']]

# Display the result
launch_sites_df
```

Out[7]:

	Launch Site	Lat	Long
0	CCAFS LC-40	28.562302	-80.577356
1	CCAFS SLC-40	28.563197	-80.576820
2	KSC LC-39A	28.573255	-80.646895
3	VAFB SLC-4E	34.632834	-120.610745

```
In [1]: import folium

# NASA Johnson Space Center coordinates
nasa_coordinate = [29.559684888503615, -95.0830971930759]

# Create the folium map centered at NASA JSC
site_map = folium.Map(location=nasa_coordinate, zoom_start=10)

# Display the map
site_map
```

Out[1]:



```
In [9]: from folium.features import DivIcon

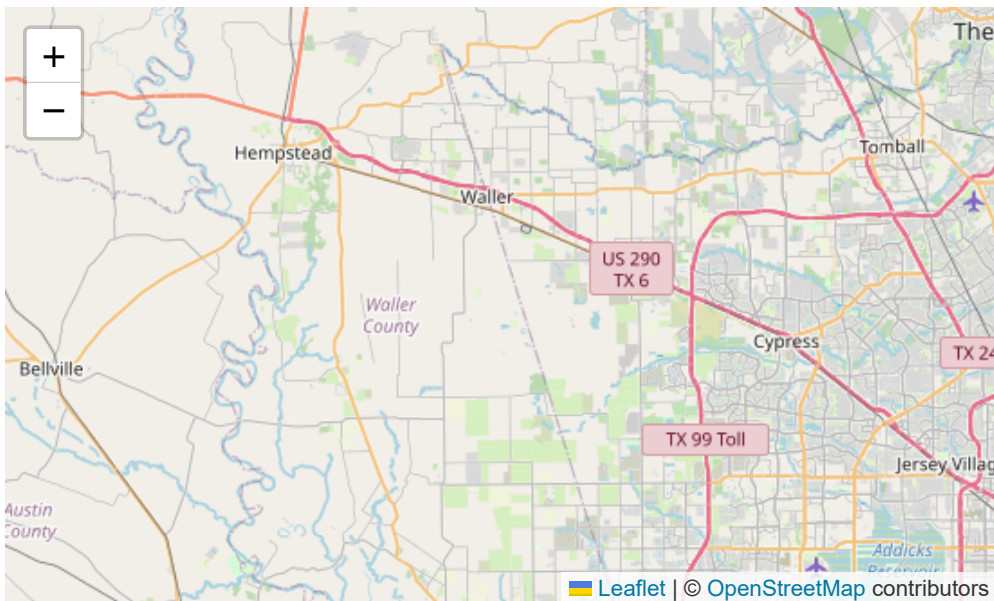
# Create a highlighted circle at NASA JSC
circle = folium.Circle(
    location=nasa_coordinate,
    radius=1000, # in meters
    color='#d35400',
    fill=True
).add_child(folium.Popup('NASA Johnson Space Center'))

# Create a marker with a Label using DivIcon
marker = folium.map.Marker(
    location=nasa_coordinate,
    icon=DivIcon(
        icon_size=(20, 20),
        icon_anchor=(0, 0),
        html='<div style="font-size: 12px; color:#d35400;"><b>NASA JSC</b></div>'
    )
)

# Add the circle and marker to the map
site_map.add_child(circle)
site_map.add_child(marker)
```

```
# Show the map
site_map
```

Out[9]:



```
In [9]: from folium.features import DivIcon

# Create a highlighted circle at NASA JSC
circle = folium.Circle(
    location=nasa_coordinate,
    radius=1000, # in meters
    color='#d35400',
    fill=True
).add_child(folium.Popup('NASA Johnson Space Center'))

# Create a marker with a Label using DivIcon
marker = folium.map.Marker(
    location=nasa_coordinate,
    icon=DivIcon(
        icon_size=(20, 20),
        icon_anchor=(0, 0),
        html='<div style="font-size: 12px; color:#d35400;"><b>NASA JSC</b></div>'
    )
)

# Add the circle and marker to the map
site_map.add_child(circle)
site_map.add_child(marker)

# Show the map
site_map
```

Out[9]:



```
In [10]: # Re-initialize the map centered at NASA JSC with a wider zoom
site_map = folium.Map(location=nasa_coordinate, zoom_start=5)

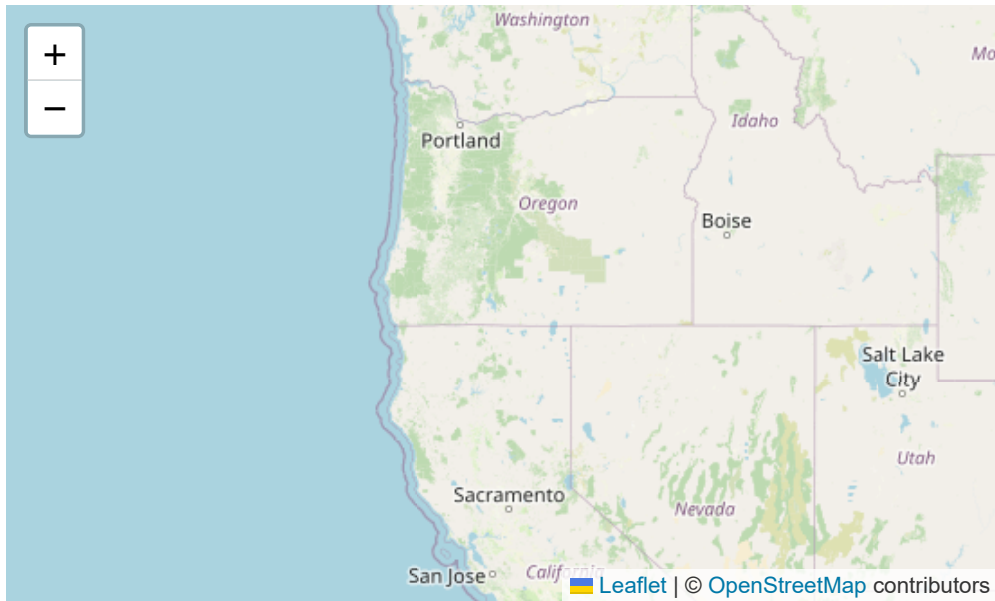
# Loop through each Launch site and add a circle and a Label marker
for index, row in launch_sites_df.iterrows():
    launch_site = row['Launch Site']
    coordinate = [row['Lat'], row['Long']]

    # Add circle to map
    folium.Circle(
        location=coordinate,
        radius=1000,
        color='#000000',
        fill=True
    ).add_child(folium.Popup(launch_site)).add_to(site_map)

    # Add marker with text Label
    folium.map.Marker(
        location=coordinate,
        icon=DivIcon(
            icon_size=(20, 20),
            icon_anchor=(0, 0),
            html='<div style="font-size: 12px; color:#d35400;"><b>%s</b></div>'
        )
    ).add_to(site_map)

# Display the map
site_map
```


Out[10]:



In [11]:

```

# Re-initialize the map with a broader view
site_map = folium.Map(location=nasa_coordinate, zoom_start=5)

# Add site Labels and circles (as before)
for index, row in launch_sites_df.iterrows():
    launch_site = row['Launch Site']
    coordinate = [row['Lat'], row['Long']]

    # Circle
    folium.Circle(
        location=coordinate,
        radius=1000,
        color='#000000',
        fill=True
    ).add_child(folium.Popup(launch_site)).add_to(site_map)

# Label marker
folium.Marker(
    location=coordinate,
    icon=DivIcon(
        icon_size=(20, 20),
        icon_anchor=(0, 0),
        html='<div style="font-size: 12px; color:#d35400;"><b>%s</b></div>'
    )
).add_to(site_map)

# Add a marker for each individual Launch – color-coded by success
for index, row in spacex_df.iterrows():
    launch_location = [row['Lat'], row['Long']]
    outcome = row['class']
    launch_label = row['Launch Site']

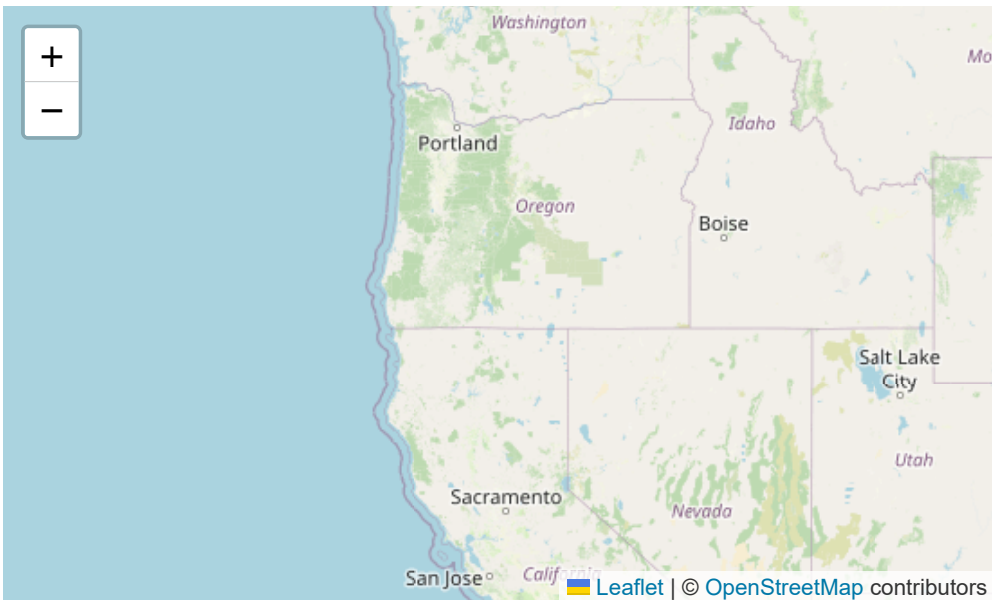
    # Color based on success/failure
    marker_color = 'green' if outcome == 1 else 'red'
    popup_text = f"{launch_label} - {'Success' if outcome == 1 else 'Failure'}"

```

```
folium.Marker(
    location=launch_location,
    popup=popup_text,
    icon=folium.Icon(color=marker_color, icon='rocket', prefix='fa')
).add_to(site_map)
```

```
# Display the map
site_map
```

Out[11]:



```
In [12]: # Calculate success rate = mean of 'class' column (1=success, 0=failure)
site_success_df = spacex_df.groupby('Launch Site')['class'].mean().reset_index()
site_success_df.columns = ['Launch Site', 'Success Rate']

# Merge with launch_sites_df to get coordinates + success rate
launch_sites_with_success = pd.merge(launch_sites_df, site_success_df, on='Launch Site')
launch_sites_with_success
```

Out[12]:

	Launch Site	Lat	Long	Success Rate
0	CCAFS LC-40	28.562302	-80.577356	0.269231
1	CCAFS SLC-40	28.563197	-80.576820	0.428571
2	KSC LC-39A	28.573255	-80.646895	0.769231
3	VAFB SLC-4E	34.632834	-120.610745	0.400000

```
In [13]: # Create a new map
site_map = folium.Map(location=nasa_coordinate, zoom_start=5)

# Loop through the sites with success rates
for index, row in launch_sites_with_success.iterrows():
    coordinate = [row['Lat'], row['Long']]
    launch_site = row['Launch Site']
```



```

success_rate = row['Success Rate'] * 100 # Convert to percentage

popup_text = f"""
<b>{launch_site}</b><br>
Success Rate: {success_rate:.1f}%
"""

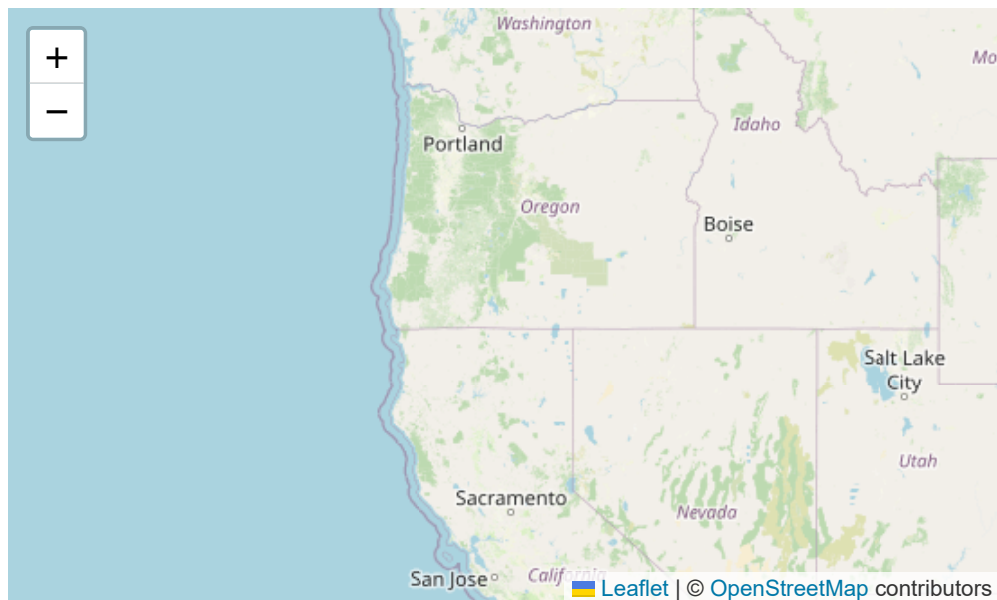
# Add circle
folium.Circle(
    location=coordinate,
    radius=1000,
    color='#3186cc',
    fill=True,
    fill_opacity=0.6
).add_child(folium.Popup(popup_text)).add_to(site_map)

# Add Label with site name
folium.Marker(
    location=coordinate,
    icon=DivIcon(
        icon_size=(20,20),
        icon_anchor=(0,0),
        html=f'<div style="font-size: 12px; color: #3186cc;"><b>{launch_site
    )
).add_to(site_map)

# Show the map
site_map

```

Out[13]:



```

In [14]: import folium
from folium.plugins import MarkerCluster
from folium.features import DivIcon

# Re-initialize the map with a broader view
site_map = folium.Map(location=nasa_coordinate, zoom_start=5)

```

```
# Add site labels and circles (as before)
for index, row in launch_sites_df.iterrows():
    launch_site = row['Launch Site']
    coordinate = [row['Lat'], row['Long']]

    # Circle for each launch site
    folium.Circle(
        location=coordinate,
        radius=1000,
        color='#3186cc',
        fill=True,
        fill_opacity=0.6
    ).add_child(folium.Popup(launch_site)).add_to(site_map)

# Label for each launch site using DivIcon
folium.Marker(
    location=coordinate,
    icon=DivIcon(
        icon_size=(20, 20),
        icon_anchor=(0, 0),
        html=f'<div style="font-size: 12px; color: #3186cc;"><b>{launch_site}</b>
    )
).add_to(site_map)

# Create a MarkerCluster object to reduce clutter
marker_cluster = MarkerCluster().add_to(site_map)

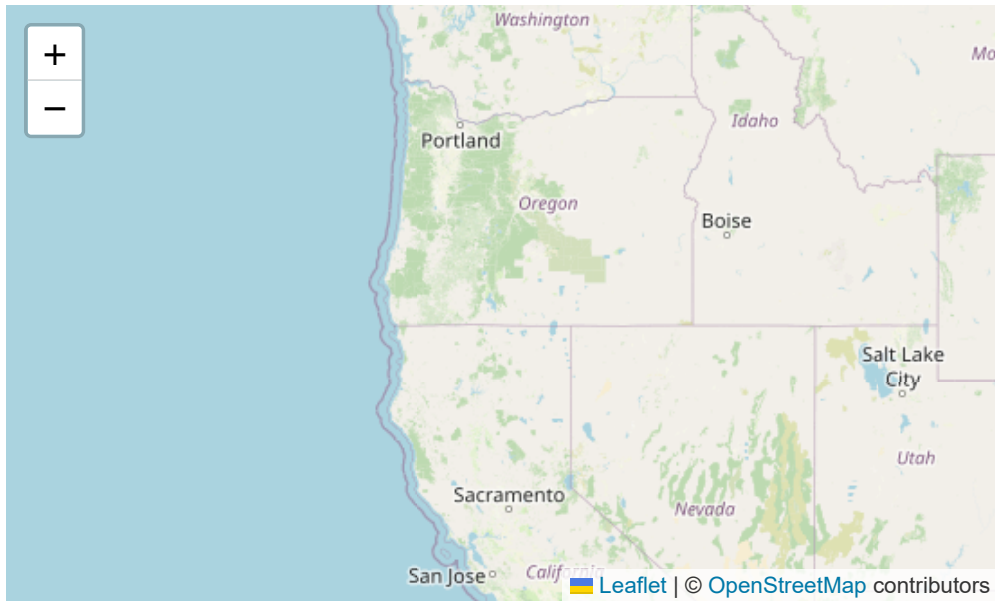
# Add a marker for each individual launch - color-coded by success
for index, row in spacex_df.iterrows():
    launch_location = [row['Lat'], row['Long']]
    outcome = row['class']
    launch_label = row['Launch Site']

    # Color the marker based on success/failure
    marker_color = 'green' if outcome == 1 else 'red'
    popup_text = f'{launch_label} - {'Success' if outcome == 1 else 'Failure'}'

    # Add each Launch marker to the MarkerCluster
    folium.Marker(
        location=launch_location,
        popup=popup_text,
        icon=folium.Icon(color=marker_color, icon='rocket', prefix='fa')
    ).add_to(marker_cluster)

# Display the map
site_map
```

Out[14]:



In [15]:

```

from folium.plugins import MarkerCluster
import folium

# Re-initialize the map with a broader view
site_map = folium.Map(location=nasa_coordinate, zoom_start=5)

# Create a MarkerCluster object
marker_cluster = MarkerCluster().add_to(site_map)

# Loop through each Launch record in spacex_df
for index, row in spacex_df.iterrows():
    launch_location = [row['Lat'], row['Long']]
    outcome = row['class']
    launch_label = row['Launch Site']

    # Determine the color of the marker based on success or failure
    marker_color = 'green' if outcome == 1 else 'red'

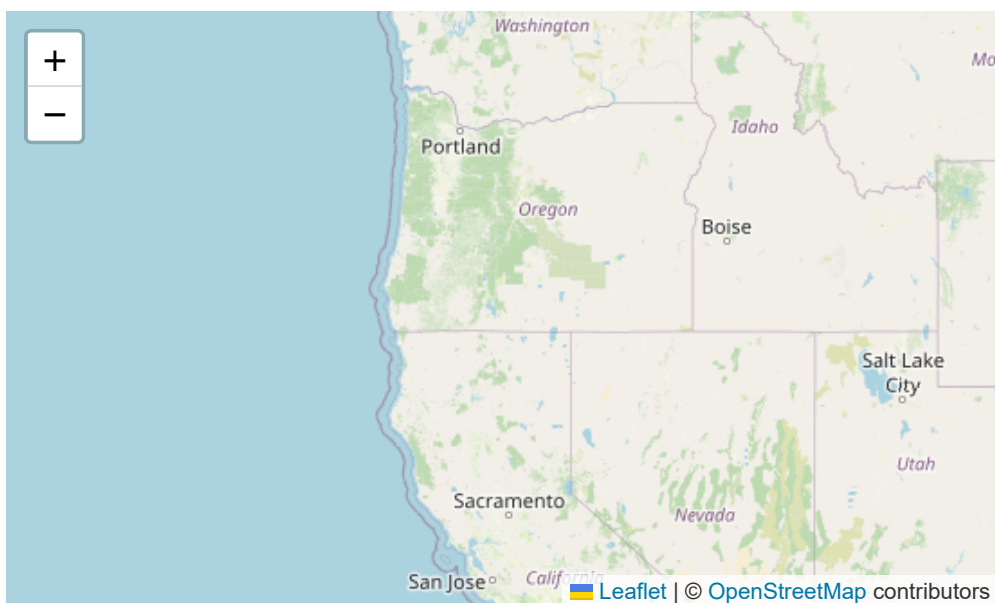
    # Create popup text for each Launch record
    popup_text = f"{launch_label} - {'Success' if outcome == 1 else 'Failure'}"

    # Add the Launch marker to the MarkerCluster
    folium.Marker(
        location=launch_location,
        popup=popup_text,
        icon=folium.Icon(color=marker_color, icon='rocket', prefix='fa')
    ).add_to(marker_cluster)

# Display the map
site_map

```

Out[15]:



In [16]:

```
# Function to assign color based on launch success
def get_marker_color(launch_class):
    return 'green' if launch_class == 1 else 'red'

# Apply the function to create the new 'marker_color' column
spacex_df['marker_color'] = spacex_df['class'].apply(get_marker_color)

# Display the updated DataFrame with the new column
spacex_df[['Launch Site', 'Lat', 'Long', 'class', 'marker_color']].tail(10)
```

Out[16]:

	Launch Site	Lat	Long	class	marker_color
46	KSC LC-39A	28.573255	-80.646895	1	green
47	KSC LC-39A	28.573255	-80.646895	1	green
48	KSC LC-39A	28.573255	-80.646895	1	green
49	CCAFS SLC-40	28.563197	-80.576820	1	green
50	CCAFS SLC-40	28.563197	-80.576820	1	green
51	CCAFS SLC-40	28.563197	-80.576820	0	red
52	CCAFS SLC-40	28.563197	-80.576820	0	red
53	CCAFS SLC-40	28.563197	-80.576820	0	red
54	CCAFS SLC-40	28.563197	-80.576820	1	green
55	CCAFS SLC-40	28.563197	-80.576820	0	red

In [17]:

```
# Re-initialize the map with a broader view
site_map = folium.Map(location=nasa_coordinate, zoom_start=5)

# Create a MarkerCluster object
```

```

marker_cluster = MarkerCluster().add_to(site_map)

# Loop through each launch record in spacex_df to create markers
for index, record in spacex_df.iterrows():
    # Coordinates for the marker
    launch_location = [record['Lat'], record['Long']]
    # Marker color based on the 'marker_color' column
    marker_color = record['marker_color']
    # Popup text to show the launch site and the success/failure
    popup_text = f"{record['Launch Site']} - {'Success' if record['class'] == 1}

    # Create the marker with customized icon based on success/failure
    marker = folium.Marker(
        location=launch_location,
        popup=popup_text,
        icon=folium.Icon(color='white', icon_color=marker_color, icon='rocket',
    )

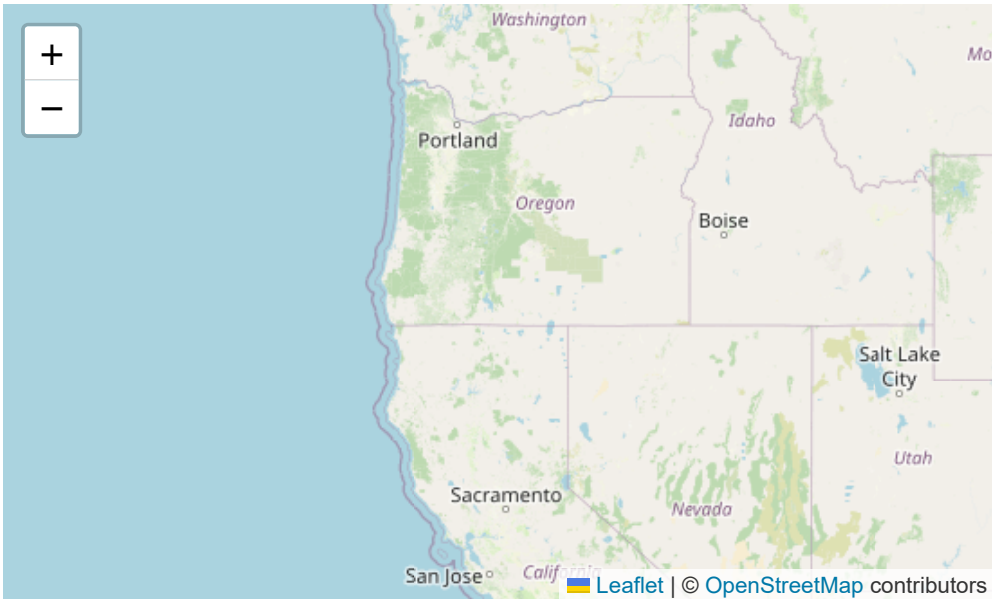
    # Add the marker to the MarkerCluster
    marker_cluster.add_child(marker)

# Add the marker_cluster to the site_map
site_map.add_child(marker_cluster)

# Display the map
site_map

```

Out[17]:



```

In [19]: import folium
from folium.plugins import MarkerCluster, MousePosition
from folium.features import DivIcon
import pandas as pd

# Load the dataset (example path, replace with actual data path)
# spacex_df = pd.read_csv('path_to_spacex_launch_geo.csv')

# Sample data initialization (for demonstration purposes)

```

```
# Assuming spacex_df has columns: Launch Site, Lat, Long, class (0 = Failure, 1
spacex_df = pd.DataFrame({
    'Launch Site': ['KSC LC-39A', 'VAFB SLC-4E', 'CCAFS LC-40', 'VAFB SLC-4E'],
    'Lat': [28.5733, 34.6321, 28.5623, 34.6321],
    'Long': [-80.6469, -120.6106, -80.5774, -120.6106],
    'class': [1, 0, 1, 0]
})

# Create a new column for marker color based on class value
spacex_df['marker_color'] = spacex_df['class'].apply(lambda x: 'green' if x == 0 :

# Define NASA Johnson Space Center coordinates
nasa_coordinate = [29.559684888503615, -95.0830971930759]

# Initialize the map
site_map = folium.Map(location=nasa_coordinate, zoom_start=5)

# Create the MousePosition plugin to show coordinates on mouse hover
formatter = "function(num) {return L.Util.formatNum(num, 5)};"
mouse_position = MousePosition(
    position='topright',
    separator=' Long: ',
    empty_string='NaN',
    lng_first=False,
    num_digits=5,
    prefix='Lat:',
    lat_formatter=formatter,
    lng_formatter=formatter,
)

site_map.add_child(mouse_position)

# Create a MarkerCluster object
marker_cluster = MarkerCluster().add_to(site_map)

# Add markers for each Launch site
for index, record in spacex_df.iterrows():
    launch_location = [record['Lat'], record['Long']]
    marker_color = record['marker_color']
    popup_text = f"{record['Launch Site']} - {'Success' if record['class'] == 1

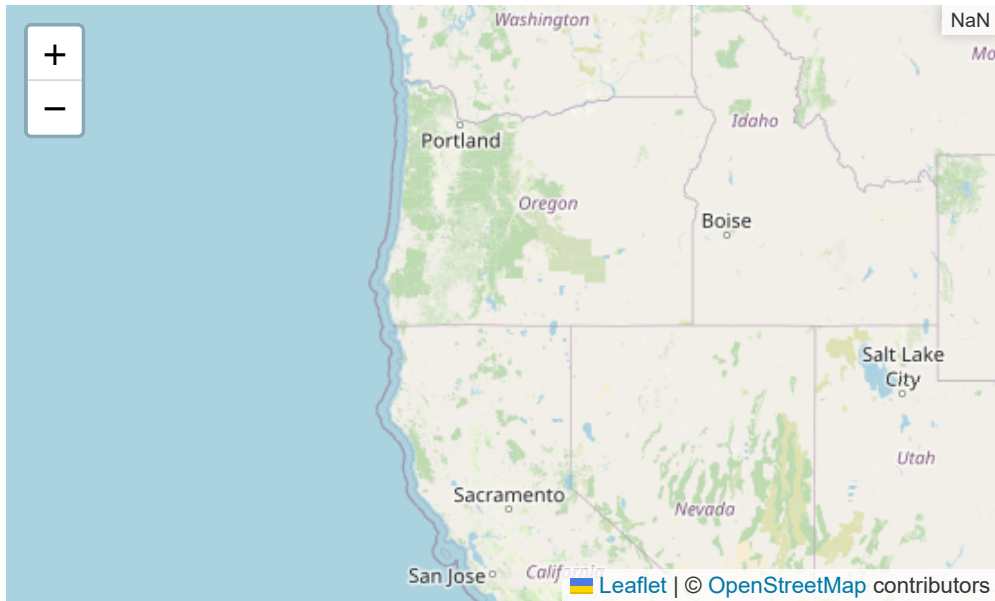
# Create the marker with customized icon
marker = folium.Marker(
    location=launch_location,
    popup=popup_text,
    icon=folium.Icon(color='white', icon_color=marker_color, icon='rocket',
)

# Add the marker to the MarkerCluster
marker_cluster.add_child(marker)

# Add the marker_cluster to the site_map
site_map.add_child(marker_cluster)

# Display the map
site_map
```


Out[19]:



In [20]:

```
# Example coordinates for the launch site (e.g., KSC LC-39A)
launch_site_lat = 28.5733
launch_site_lon = -80.6469

# Example: You find a railway or coastline and get the coordinates (Let's assume)
# These are just sample coordinates for illustration; you should replace them with actual data
railway_lat = 28.5750 # Sample latitude for a railway or coastline
railway_lon = -80.6400 # Sample longitude for a railway or coastline

# Calculate the distance between the launch site and the railway
distance = calculate_distance(launch_site_lat, launch_site_lon, railway_lat, railway_lon)

# Print the result in kilometers
print(f"Distance from launch site to railway: {distance:.2f} km")
```

NameError

Traceback (most recent call last)

Cell In[20], line 11

```
8 railway_lon = -80.6400 # Sample longitude for a railway or coastline
10 # Calculate the distance between the launch site and the railway
--> 11 distance = calculate_distance(launch_site_lat, launch_site_lon, railway_lat, railway_lon)
13 # Print the result in kilometers
14 print(f"Distance from launch site to railway: {distance:.2f} km")
```

NameError: name 'calculate_distance' is not defined

In [21]:

```
# Import necessary functions from math
from math import sin, cos, sqrt, atan2, radians

# Define the function to calculate distance using the Haversine formula
def calculate_distance(lat1, lon1, lat2, lon2):
    # approximate radius of earth in km
    R = 6373.0
```

```

# Convert Latitude and Longitude from degrees to radians
lat1 = radians(lat1)
lon1 = radians(lon1)
lat2 = radians(lat2)
lon2 = radians(lon2)

# Calculate the difference in longitudes and latitudes
dlon = lon2 - lon1
dlat = lat2 - lat1

# Apply the Haversine formula
a = sin(dlat / 2)**2 + cos(lat1) * cos(lat2) * sin(dlon / 2)**2
c = 2 * atan2(sqrt(a), sqrt(1 - a))

# Distance in kilometers
distance = R * c
return distance

# Example: Coordinates for a launch site and a nearby railway
launch_site_lat = 28.5733 # Latitude of KSC LC-39A
launch_site_lon = -80.6469 # Longitude of KSC LC-39A

# Example: Coordinates of a nearby railway (replace these with real coordinates)
railway_lat = 28.5750 # Sample latitude for the railway
railway_lon = -80.6400 # Sample longitude for the railway

# Calculate the distance between the launch site and the railway
distance = calculate_distance(launch_site_lat, launch_site_lon, railway_lat, railway_lon)

# Print the result
print(f"Distance from launch site to railway: {distance:.2f} km")

```

Distance from launch site to railway: 0.70 km

```

In [22]: import folium
from folium.plugins import MousePosition
from folium.features import DivIcon
from math import sin, cos, sqrt, atan2, radians

# Define the Haversine formula to calculate the distance
def calculate_distance(lat1, lon1, lat2, lon2):
    R = 6373.0 # Radius of the Earth in kilometers

    # Convert degrees to radians
    lat1 = radians(lat1)
    lon1 = radians(lon1)
    lat2 = radians(lat2)
    lon2 = radians(lon2)

    # Difference in coordinates
    dlon = lon2 - lon1
    dlat = lat2 - lat1

    # Apply the Haversine formula
    a = sin(dlat / 2)**2 + cos(lat1) * cos(lat2) * sin(dlon / 2)**2
    c = 2 * atan2(sqrt(a), sqrt(1 - a))

```

```
# Calculate distance
distance = R * c
return distance

# Example coordinates for the launch site (KSC LC-39A)
launch_site_lat = 28.5733
launch_site_lon = -80.6469

# Example coordinates for the coastline (found using MousePosition)
coastline_lat = 28.56367 # Replace with your coastline Latitude
coastline_lon = -80.57163 # Replace with your coastline Longitude

# Calculate the distance from the launch site to the coastline
distance_coastline = calculate_distance(launch_site_lat, launch_site_lon, coastline_lat, coastline_lon)

# Initialize the map centered on the launch site
site_map = folium.Map(location=[launch_site_lat, launch_site_lon], zoom_start=10)

# Add the MousePosition plugin to the map to get coordinates on hover
formatter = "function(num) {return L.Util.formatNum(num, 5)};"
mouse_position = MousePosition(
    position='topright',
    separator=' Long: ',
    empty_string='NaN',
    lng_first=False,
    num_digits=5,
    prefix='Lat:',
    lat_formatter=formatter,
    lng_formatter=formatter,
)

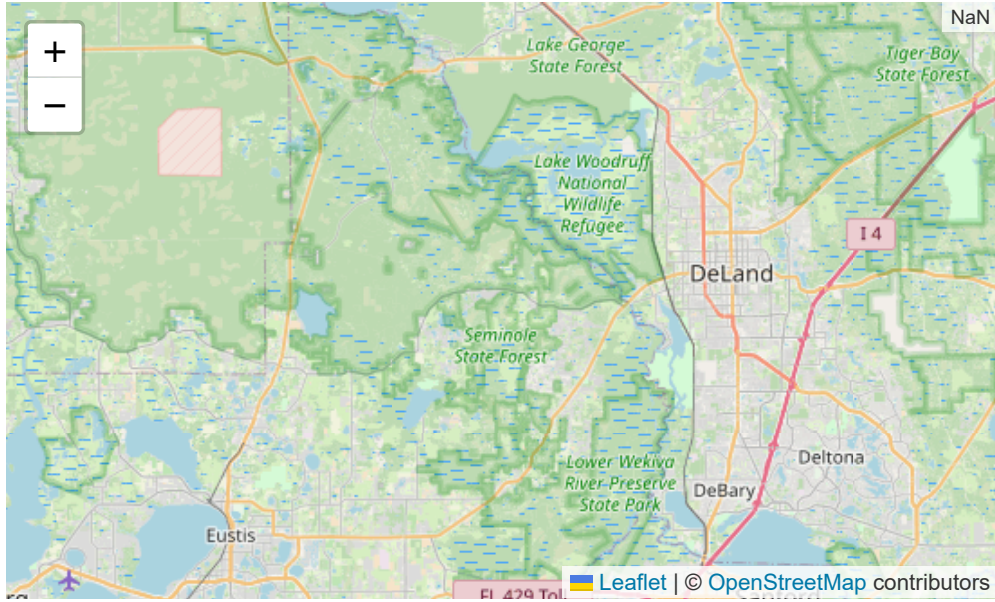
site_map.add_child(mouse_position)

# Create and add a folium.Marker on the coastline point
coastline_marker = folium.Marker(
    location=[coastline_lat, coastline_lon],
    popup=f"Coastline Point: {coastline_lat}, {coastline_lon}",
    icon=folium.Icon(color='blue', icon='info-sign')
).add_to(site_map)

# Display the distance to the coastline on the marker icon
distance_marker = folium.Marker(
    location=[coastline_lat, coastline_lon],
    icon=DivIcon(
        icon_size=(20, 20),
        icon_anchor=(0, 0),
        html=f'<div style="font-size: 12px; color:#d35400;"><b>{distance_coastline}</b></div>'
    )
).add_to(site_map)

# Display the map
site_map
```

Out[22]:



In [24]:

```
import folium
from folium.plugins import MousePosition
from folium.features import DivIcon
from math import sin, cos, sqrt, atan2, radians

# Define the Haversine formula to calculate the distance
def calculate_distance(lat1, lon1, lat2, lon2):
    R = 6373.0 # Radius of the Earth in kilometers

    # Convert degrees to radians
    lat1 = radians(lat1)
    lon1 = radians(lon1)
    lat2 = radians(lat2)
    lon2 = radians(lon2)

    # Difference in coordinates
    dlon = lon2 - lon1
    dlat = lat2 - lat1

    # Apply the Haversine formula
    a = sin(dlat / 2)**2 + cos(lat1) * cos(lat2) * sin(dlon / 2)**2
    c = 2 * atan2(sqrt(a), sqrt(1 - a))

    # Calculate distance
    distance = R * c
    return distance

# Example coordinates for the launch site (KSC LC-39A)
launch_site_lat = 28.5733
launch_site_lon = -80.6469

# Example coordinates for the coastline (found using MousePosition)
coastline_lat = 28.56367 # Replace with your coastline Latitude
coastline_lon = -80.57163 # Replace with your coastline Longitude
```

```
# Calculate the distance from the launch site to the coastline
distance_coastline = calculate_distance(launch_site_lat, launch_site_lon, coastl

# Initialize the map centered on the launch site
site_map = folium.Map(location=[launch_site_lat, launch_site_lon], zoom_start=10)

# Add the MousePosition plugin to the map to get coordinates on hover
formatter = "function(num) {return L.Util.formatNum(num, 5)};"
mouse_position = MousePosition(
    position='topright',
    separator=' Long: ',
    empty_string='NaN',
    lng_first=False,
    num_digits=5,
    prefix='Lat:',
    lat_formatter=formatter,
    lng_formatter=formatter,
)

site_map.add_child(mouse_position)

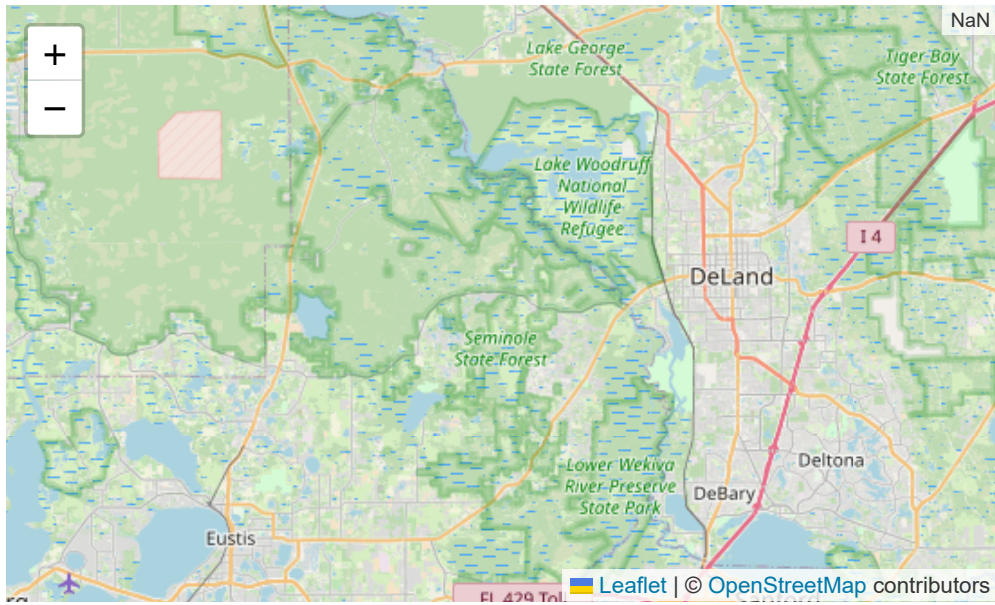
# Create and add a folium.Marker on the coastline point
coastline_marker = folium.Marker(
    location=[coastline_lat, coastline_lon],
    popup=f"Coastline Point: {coastline_lat}, {coastline_lon}",
    icon=folium.Icon(color='blue', icon='info-sign')
).add_to(site_map)

# Display the distance to the coastline on the marker icon
distance_marker = folium.Marker(
    location=[coastline_lat, coastline_lon],
    icon=DivIcon(
        icon_size=(20, 20),
        icon_anchor=(0, 0),
        html=f'<div style="font-size: 12px; color:#d35400;"><b>{distance_coastl
    )
).add_to(site_map)

# Create a Polyline between the launch site and coastline
coordinates = [(launch_site_lat, launch_site_lon), (coastline_lat, coastline_lon)]
lines = folium.Polyline(locations=coordinates, weight=2, color='blue').add_to(site_map)

# Display the map
site_map
```

Out[24]:



In [26]:

```
import folium
from folium.plugins import MousePosition
from folium.features import DivIcon
from math import sin, cos, sqrt, atan2, radians

# Define the Haversine formula to calculate the distance
def calculate_distance(lat1, lon1, lat2, lon2):
    R = 6373.0 # Radius of the Earth in kilometers

    # Convert degrees to radians
    lat1 = radians(lat1)
    lon1 = radians(lon1)
    lat2 = radians(lat2)
    lon2 = radians(lon2)

    # Difference in coordinates
    dlon = lon2 - lon1
    dlat = lat2 - lat1

    # Apply the Haversine formula
    a = sin(dlat / 2)**2 + cos(lat1) * cos(lat2) * sin(dlon / 2)**2
    c = 2 * atan2(sqrt(a), sqrt(1 - a))

    # Calculate distance
    distance = R * c
    return distance

# Example coordinates for the launch site (KSC LC-39A)
launch_site_lat = 28.5733
launch_site_lon = -80.6469

# Example coordinates for a nearby point of interest (e.g., a city, railway, highway)
# Replace these with real coordinates from the map or MousePosition plugin
poi_lat = 28.5300 # Example: City or point of interest Latitude
poi_lon = -80.6200 # Example: City or point of interest Longitude
```

```
# Calculate the distance from the launch site to the point of interest
distance_poi = calculate_distance(launch_site_lat, launch_site_lon, poi_lat, poi_lon)

# Initialize the map centered on the launch site
site_map = folium.Map(location=[launch_site_lat, launch_site_lon], zoom_start=10)

# Add the MousePosition plugin to the map to get coordinates on hover
formatter = "function(num) {return L.Util.formatNum(num, 5)};"
mouse_position = MousePosition(
    position='topright',
    separator=' Long: ',
    empty_string='NaN',
    lng_first=False,
    num_digits=5,
    prefix='Lat:',
    lat_formatter=formatter,
    lng_formatter=formatter,
)

site_map.add_child(mouse_position)

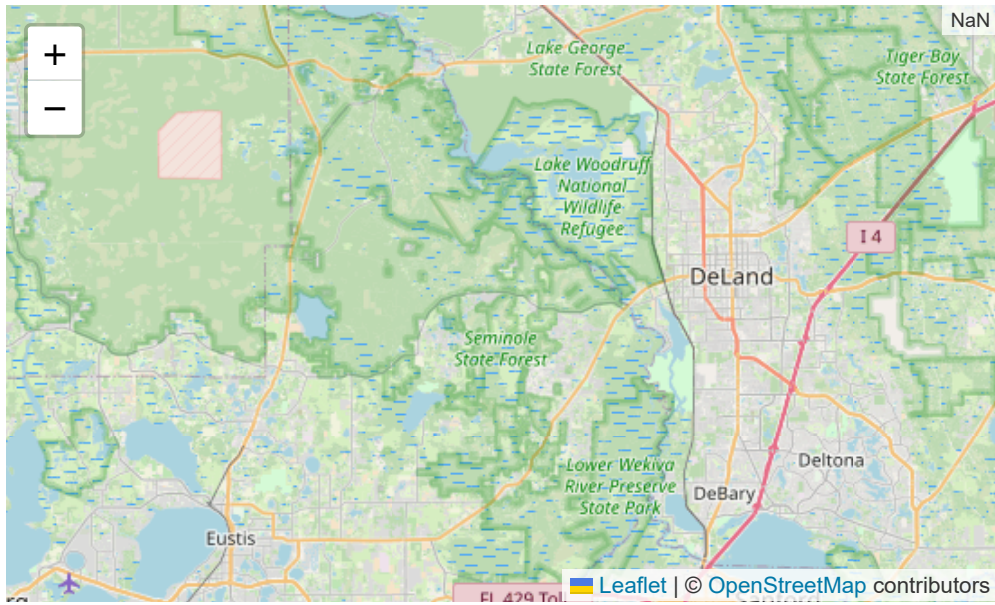
# Create and add a folium.Marker on the point of interest
poi_marker = folium.Marker(
    location=[poi_lat, poi_lon],
    popup=f"Point of Interest: {poi_lat}, {poi_lon}",
    icon=folium.Icon(color='green', icon='info-sign')
).add_to(site_map)

# Display the distance to the point of interest on the marker icon
distance_marker = folium.Marker(
    location=[poi_lat, poi_lon],
    icon=DivIcon(
        icon_size=(20, 20),
        icon_anchor=(0, 0),
        html=f'<div style="font-size: 12px; color:#d35400;"><b>{distance_poi:.2f} km</b>
    )
).add_to(site_map)

# Create a PolyLine between the launch site and the point of interest
coordinates = [(launch_site_lat, launch_site_lon), (poi_lat, poi_lon)]
line = folium.PolyLine(locations=coordinates, weight=2, color='red').add_to(site_map)

# Display the map
site_map
```


Out[26]:



In []: